Electronics Research Institute

HDL Side of The Crop Yield Prediction

Presented to Dr. Heba Draz, ECIP Internship Program Committee
hdraz@aucegypt.edu

Course: Hardware/Software Co-Design
Presented by: Ali Adel | ali.adel@email.com

https://eri.sci.eg

Agenda

01 Recap

02 Problems Encountered

03 IEEE-754 Floating-Point

04 FPGA Flow

05 Golden Model

06 Results

07 Conclusion

Problems Encountered

What exactly did we talk about last week?

Problems Encountered
Floating points

Verilog has limited abstraction.

No advanced data types as inputs, such as ‘real’ data type.
This made rapid algorithm modelling difficult.

Why SystemVerilog?

- SystemVerilog solved early-stage modeling problems.
- Benefits:
  - Supports real data type as an input.
  - Faster development of the golden model.
  - Ignore hardware limitations temporarily.

Why This Design Is NOT Synthesizable?

- Key reasons this model cannot map to hardware:
  - Use of real inputs and registers
  - Arithmetic on real numbers (+ - * /)
  - Division by real constants
  - Casting between logic and real

Problems Encountered
Floating points

Why I Must Go Back to Verilog

- To move toward actual hardware implementation:
- Design must be fully synthesizable
- Hardware only supports:
  - Integers
  - Fixed-point
  - Explicit floating-point formats

- Returning to Verilog (or synthesizable SystemVerilog)

Problems Encountered
Floating points

The Floating-Point Problem in Hardware

Key challenge:

- You cannot use real in hardware
- You cannot directly add or subtract floating-point numbers

So, the question becomes:

How do we represent real-world values in hardware?

Solutions

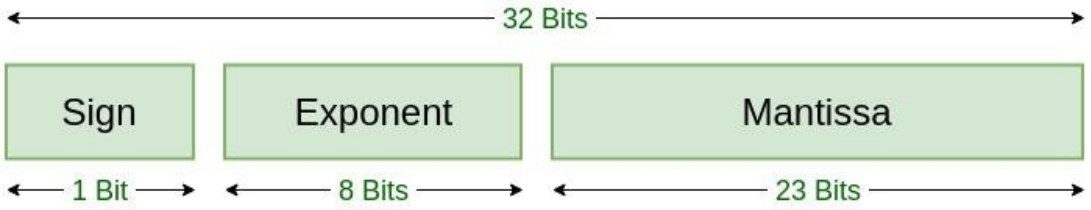
How do we represent real-world values in hardware?

IEEE-754 Floating-Point Representation

The Floating-Point Problem in Hardware

- A floating-point number is stored as:
  - Sign bit
  - Exponent
  - Mantissa (fraction)
- Common types:
  - Single precision (32-bit)
  - Double precision (64-bit)

This representation allows real-number behavior using pure binary logic.



Single Precision
IEEE 754 Floating-Point Standard

IEEE-754 Floating-Point Representation

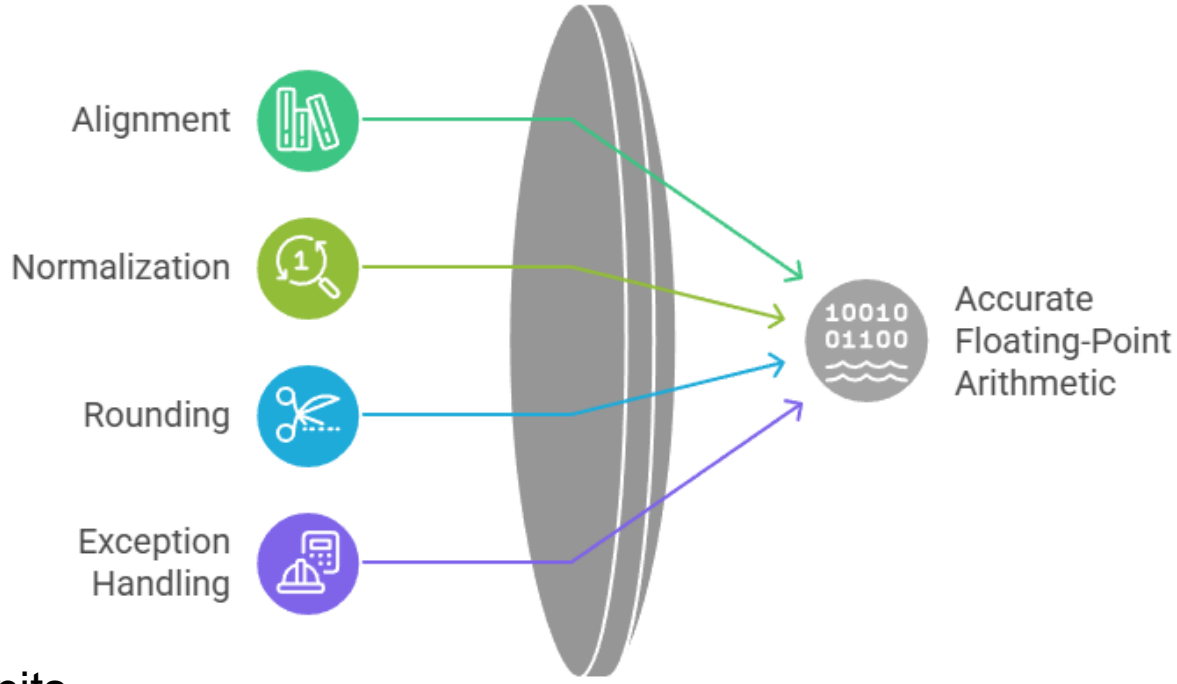
The Floating-Point Problem in Hardware

Unlike software:

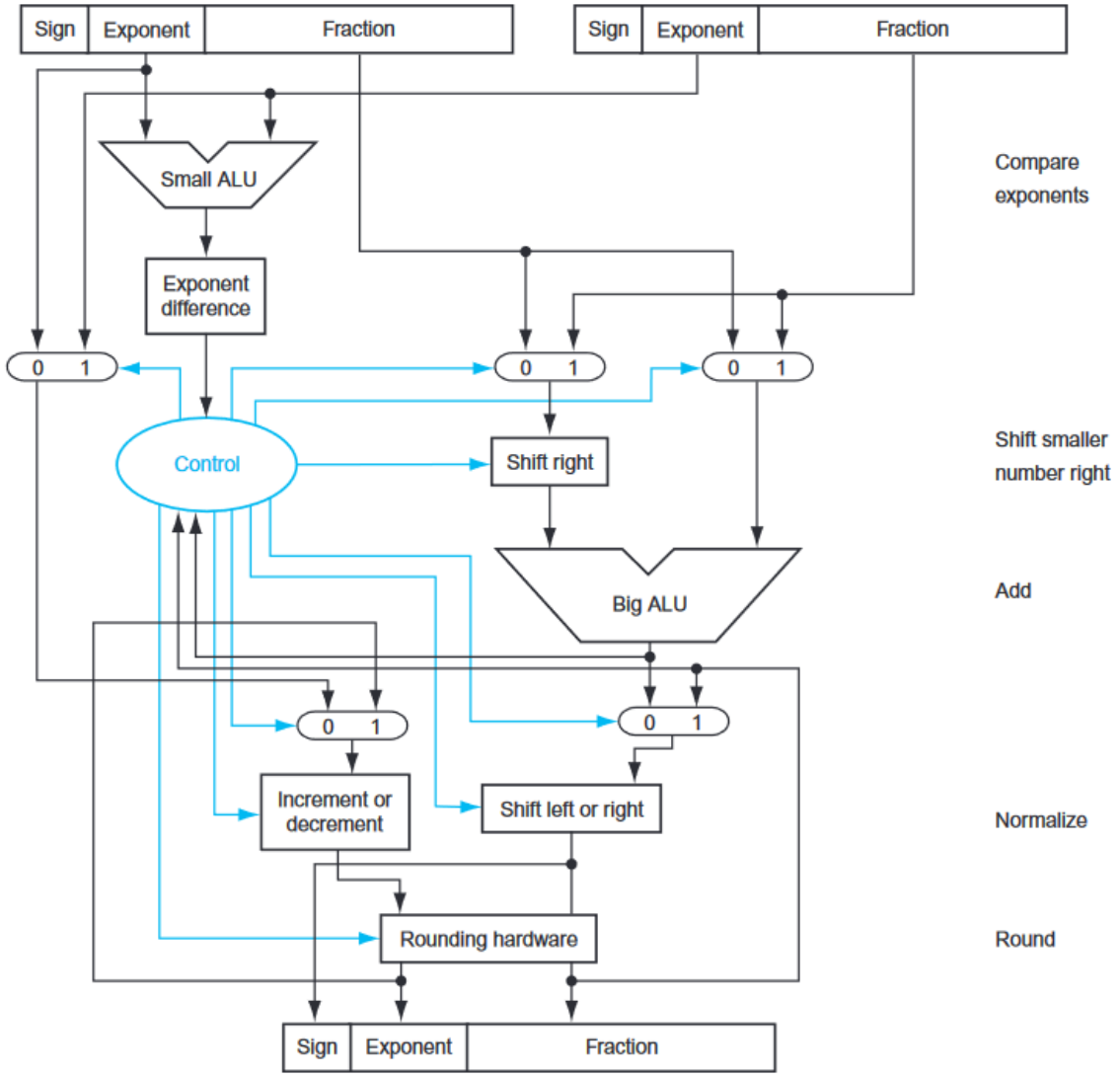
- Hardware cannot:
  - Just use + or - operators
- Floating-point arithmetic requires:
  - Alignment
  - Normalization
  - Rounding
  - Exception handling

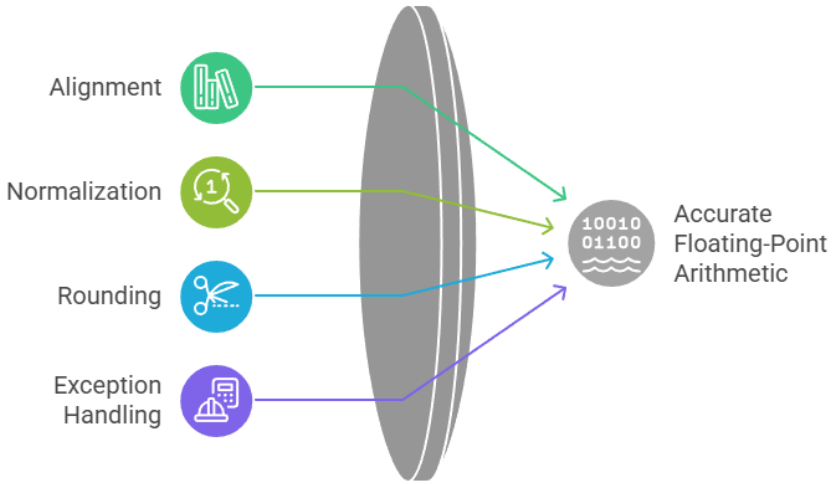
Result:

Floating-point math requires dedicated hardware units.



IEEE-754 Floating-Point Representation





FPGA Flow

High-Level Design

Python code defines the intended behavior of the digital system

Register-Transfer Level

RTL describes the hardware implementation with finite state machines and datapath

Verification Environment

UVM testbench generates stimuli and monitors RTL output

Functional Verification

Scoreboard compares RTL output against Python golden model

Hardware Deployment

Verified RTL is synthesized and deployed onto an FPGA

FPGA Flow: Register-Transfer Level

- SystemVerilog to Synthesizable Verilog

SystemVerilog Design

Non-synthesizable RTL code

IEEE 754 Conversion

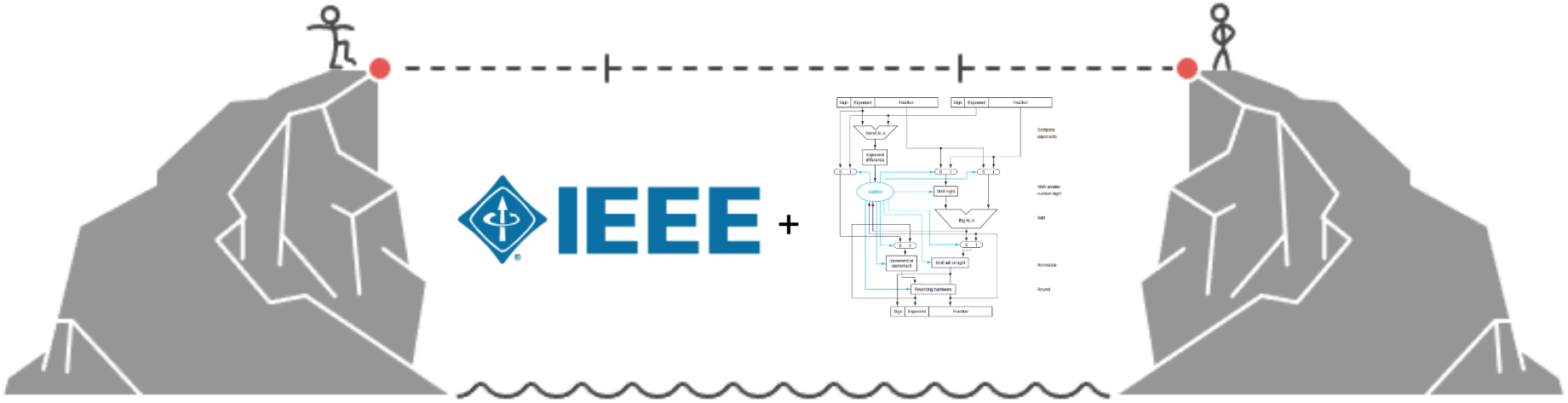
Implement floating-point format

Floating-Point Arithmetic

Add arithmetic unit

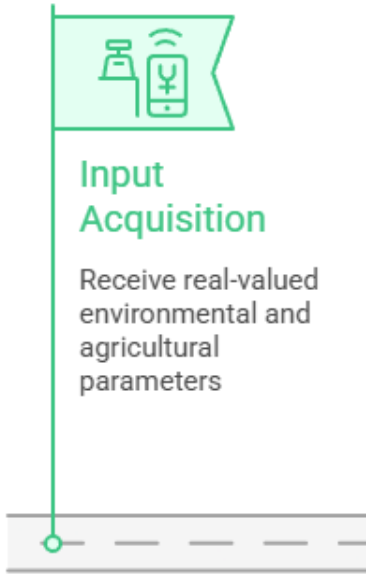
Synthesizable Verilog

Hardware-ready RTL code



FPGA Flow: SystemVerilog Golden Model

- SystemVerilog to Synthesizable Verilog

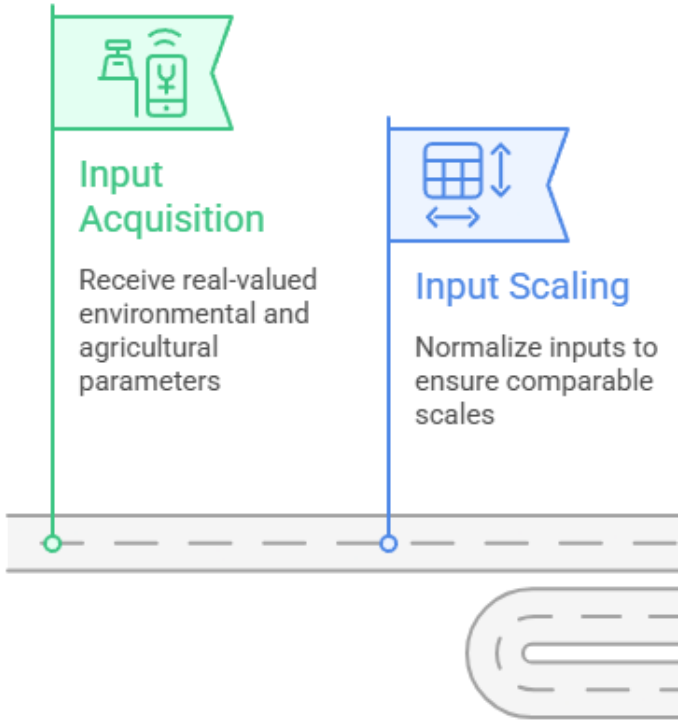


golden_model.sv

```
module golden_model (
    input  logic clk,
    input  logic rst,

    input  real soil_moisture,
    input  real soil_ph,...
```

FPGA Flow: SystemVerilog Golden Model



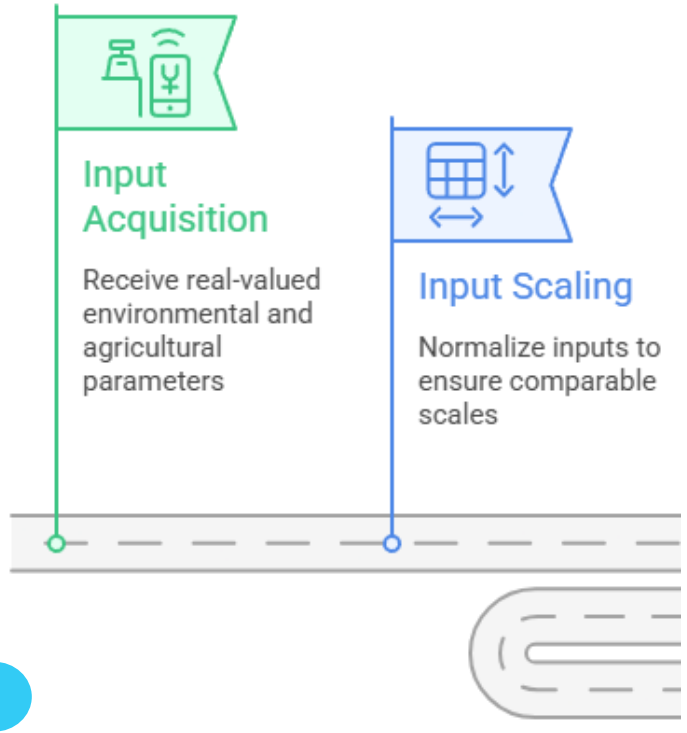
golden_model.sv

```
// ========================
//  Golden Model Parameters
// ========================
parameter real SOILMOISTURE_MEAN  = 26.75;
parameter real SOILMOISTURE_STD   = 10.15;
parameter real SOILPH_MEAN        = 6.524;
parameter real SOILPH_STD         = 0.586;...
```

FPGA Flow: SystemVerilog Golden Model

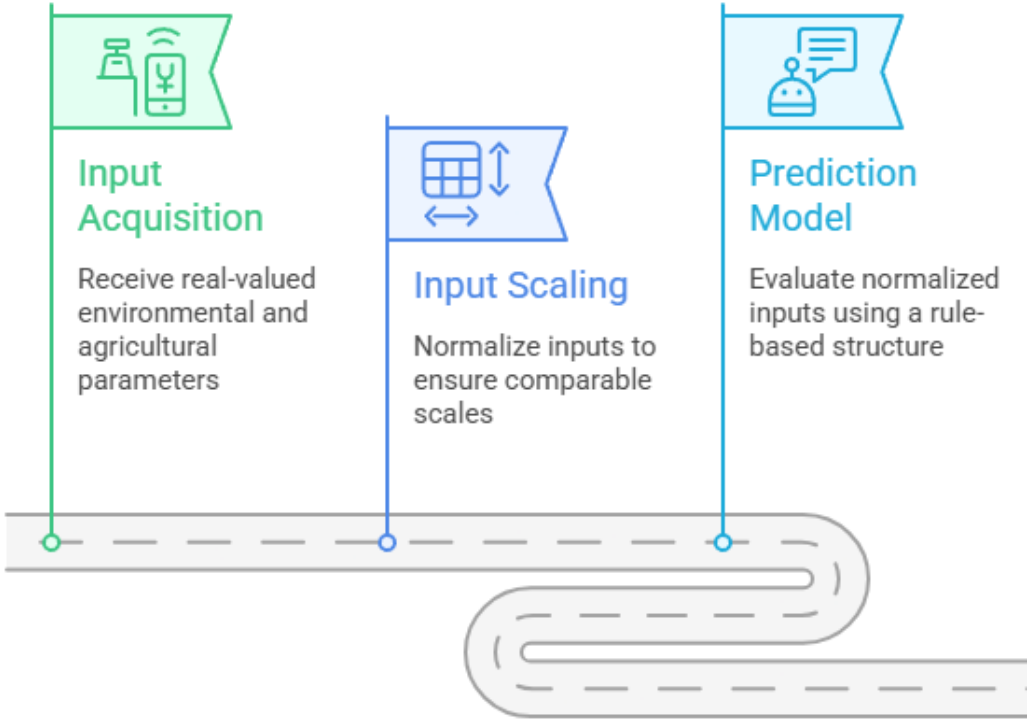
golden_model.sv

```
always_comb begin
    soil_moisture_reg  = (soil_moisture   - SOILMOISTURE_MEAN)   / SOILMOISTURE_STD;
    soil_ph_reg        = (soil_ph         - SOILPH_MEAN)         / SOILPH_STD;...
```



$$X_{new} = \frac{X_i - X_{mean}}{\text{Standard Deviation}}$$

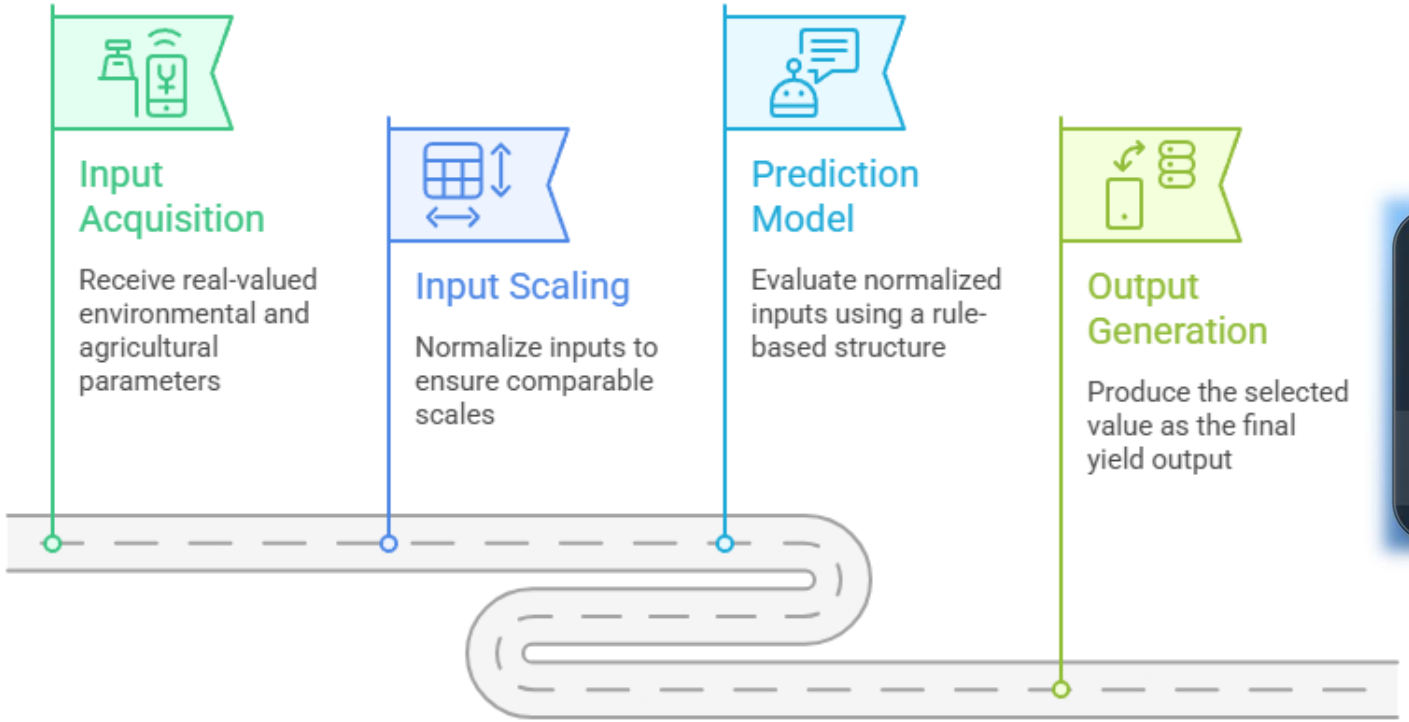
FPGA Flow: SystemVerilog Golden Model



golden_model.sv

```
// ========================
//  Prediction Logic (Golden Model)
// ========================
always_ff @(posedge clk or posedge rst) begin
  if (rst) begin
      yield_reg <= 0.0;
  end else begin
      if (soil_moisture_reg <= 0.86) begin
          if (humidity_reg <= -1.16)
              yield_reg <= 4031.19;
```

FPGA Flow: SystemVerilog Golden Model



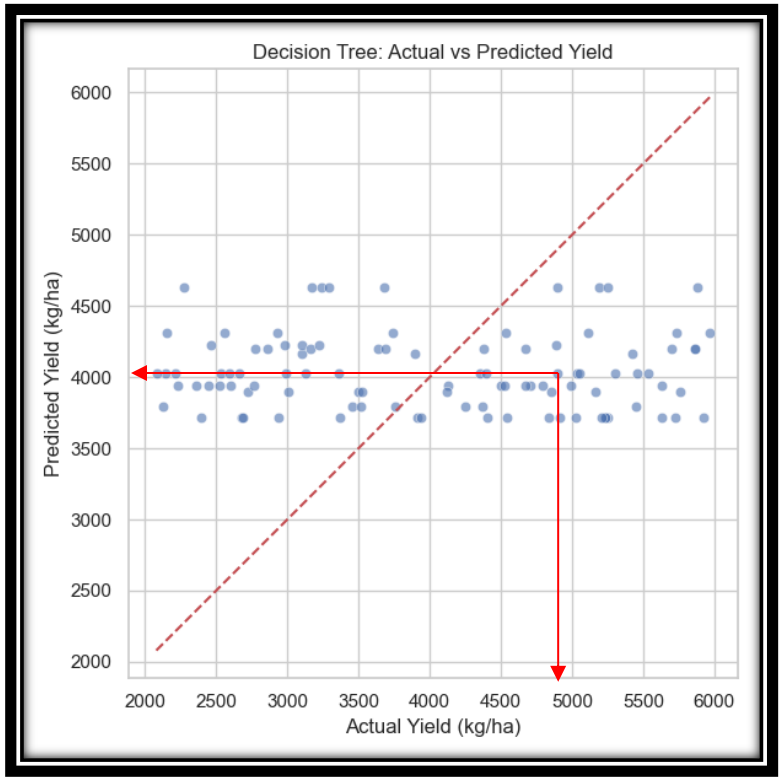
golden_model.sv

```
// ========================
//  Output Assignment
// ========================
assign yield = yield_reg;
```

Results

Initial Verilog Design results

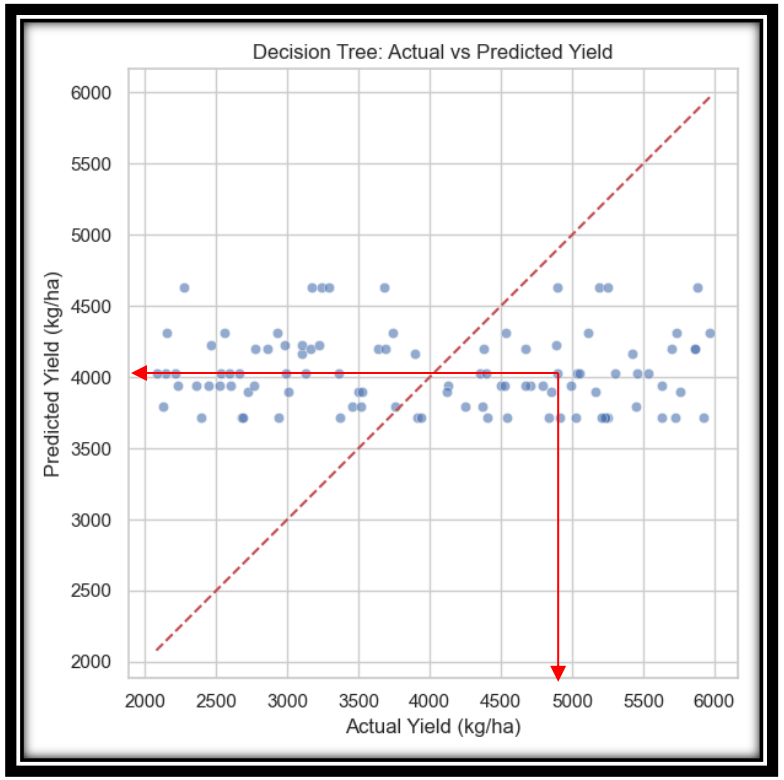
Row	Golden Model	Actual Yield	RTL Yield
1	3716.00	4408.00	4167.00
2	4167.00	5390.00	4167.00
3	4314.00	2931.00	4167.00
4	4314.00	4228.00	4167.00
5	4023.00	4980.00	4167.00
6	3716.00	4384.00	4167.00
7	3793.00	4501.00	4167.00



Results

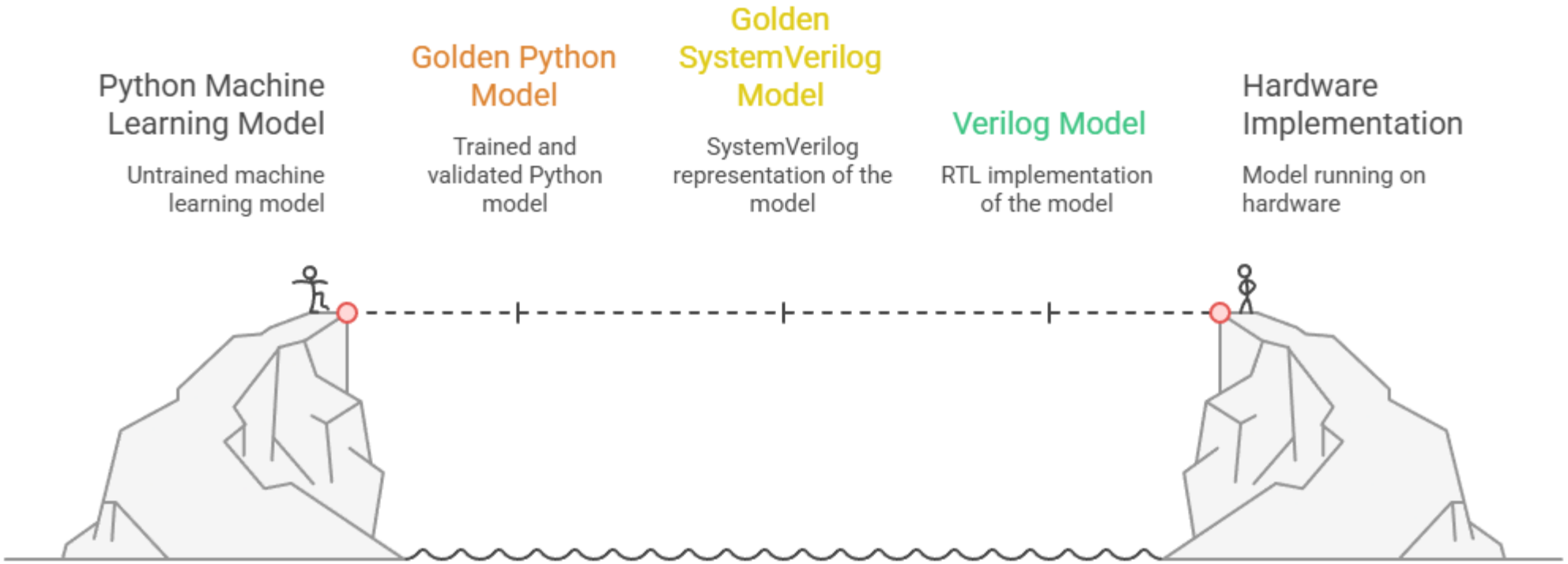
Initial SystemVerilog Design results

```
# ------------------------------------------------------------
# | Row | Golden Model   | Actual Yield   | RTL Yield          |
# ------------------------------------------------------------
# |  1  |      3716.00 |      4408.00 |      3716.00 |
# |  2  |      4167.00 |      5390.00 |      4167.00 |
# |  3  |      4314.00 |      2931.00 |      4314.00 |
# |  4  |      4314.00 |      4228.00 |      4314.00 |
# |  5  |      4023.00 |      4980.00 |      4023.00 |
# |  6  |      3716.00 |      4384.00 |      3716.00 |
# |  7  |      3793.00 |      4501.00 |      3793.00 |
# ------------------------------------------------------------
```



Conclusion

- The model is intentionally non-synthesizable and serves as a functional reference.
- Hardware implementation requires moving away from real types to explicit numeric representations.
- No need for the python implementation, currently it's all about **HDLs**



Thank you

Any Questions?